

How each part works, and where it lives

Every question from the review, answered in plain words. For each one: **what happens** (like a story), **which file does it**, and **where to change it**. Things fixed after the review are marked ☒ NEW.

How to read file names. [views/](#) = the screens you see. [api.py](#) = the waiter who carries messages between the screen and the kitchen. [servicedesk/](#) = the kitchen (the brain and the rules). [datasets/](#) = the recipe books from the brief. [servicedesk/content/](#) = our own recipe notes on top.

Chapters

1. The big picture in one minute
2. Employee side: chat, form, screenshots, replies
3. Fixed? The employee closes the ticket ☒ NEW
4. Agent side: queue, alerts, ticket page
5. Bot answers log and success rate ☒ NEW filters
6. Knowledge and knowledge gaps
7. Charts C1 to C12
8. Agent performance
9. Shift
10. Corrections log and user accounts
11. What was fixed after the review
12. Cheat sheet: I want to change...

1 The big picture in one minute

A shop with a counter, a kitchen and a diary.

Imagine a little shop. The **counter** is the screen people see. A **waiter** carries every note from the counter to the **kitchen** and back. The kitchen has a **cook** (the bot's brain) and **recipe books** (the help articles and rules). Everything that happens is written in one big **diary** (the database), and the reports are just someone counting pages in that diary.

Part of the shop	In the project	File or folder
The counter (screens)	Every page: Chat, Ticket form, My tickets, My queue, Ticket, Charts...	<code>views/*.py</code> , <code>app.py</code> (menu)
The waiter	Receives every click and message, checks who you are, calls the kitchen	<code>api.py</code>
The cook	Understands the problem, asks questions, picks the fix, decides to hand over	<code>servicedesk/orchestrator.py</code>
The cook's judge	The AI (TypeSafe Jev) only answers multiple-choice questions: which category? did it work?	<code>servicedesk/brain.py</code>
Recipe books	Categories, priority matrix, routing, help articles, form fields	<code>datasets/*.json</code> read by <code>servicedesk/knowledge.py</code>
Our recipe notes	Friendly fix steps, the bot's question wording, real SLA hours, software list	<code>servicedesk/content/*.json</code>
Helpers	Tickets, alerts, screenshots, knowledge, reports, accounts	<code>servicedesk/services/*.py</code>
The diary	tickets, messages, events, bot_answers, corrections, alerts...	<code>servicedesk/store.py</code> → <code>servicedesk.db</code>
Settings	Numbers like 2 fix attempts, 9–6 office hours, P1 re-alert every 2 minutes	<code>servicedesk/config.py</code> and <code>.env</code>

2 Employee side

Chat, the ticket form, screenshots, and replies from IT.

2.1 I type my problem. How does the bot find the answer?

Like asking a librarian. You tell her your problem in your own words. She first works out **which shelf** (category), asks **one or two small questions** if she needs to, then picks **the best book** (help article) from that shelf and reads you the easy steps.

Step	What happens	File
1	You type on the Chat page; it is sent to the waiter	views/employee_chat.py → api.py (POST /chat)
2	The cook asks the AI: what kind of problem? which category? how urgent? any danger?	orchestrator.py _trriage() → brain.py triage()
3	Priority is worked out from impact × urgency	knowledge.py compute_priority() + datasets/priority_matrix.json
4	The AI scores every help article on that shelf; the best one wins (it must score 0.50 or more)	orchestrator.py _ground_and_continue() → brain.py ground()
5	Missing details? The bot asks (max 2 questions), with buttons	servicedesk/content/field_questions.json
6	The bot sends Fix 1 in friendly words; if it fails, Fix 2; if that fails, it hands over to a person	servicedesk/content/kb_employee_drafts.json

2.2 Where are the Attempt 1 and Attempt 2 replies set? (the old “field reply”)

Same idea as before, just tidier. There are three places:

What	File	Example
The bot's questions and their buttons (this is the old “field reply”)	servicedesk/content/field_questions.json	“Which sign-in method do you use?” with buttons
The friendly Fix 1, Fix 2 and the “what happens next” text after a hand-over	servicedesk/content/kb_employee_drafts.json	attempt1 → steps, why it helps; attempt2; handoff_message
The original article written for IT	datasets/kb_articles.json	KB-006 VPN certificate error

Easiest way to change a reply: Knowledge page → open the article → edit → the supervisor approves. Employees only ever see approved text. The file is the starting copy; approved edits are kept in the diary (table kb_versions).

2.3 The dropdowns on the Ticket form: where do they come from?

Dropdown	Comes from
Category	datasets/categories.json (12 categories)
Issue type (changes with the category)	datasets/subcategories.json

Dropdown	Comes from
The extra boxes for each category	<code>datasets/dynamic_form_fields.json</code>
Who is affected? Can you keep working?	Fixed options in <code>api.py</code> (they set impact and urgency, so priority is fair)

All of these reach the screen through one call, `/meta` in `api.py`. Pressing **Review** sends the form to `orchestrator.py review_form()`, which checks the category, masks passwords and works out the priority.

2.4 Screenshots: how are they read, and how is the user told if it's wrong? ☒ NEW

A tiny robot with glasses (**RapidOCR**) reads the words in the picture. It hides anything that looks like a password, picks out the error lines and error codes, and gives them to the bot. File: `servicedesk/services/attachments.py`.

What the picture shows	What the employee is told
An error that matches the problem	"I can see error 809 in your screenshot." The bot uses it.
Text but no error	"I couldn't spot an error message in it... please type it exactly as it appears."
No text at all (a photo)	"I couldn't find any text... press Win + Shift + S, or type the message you see."
A blurry photo of a screen	"I couldn't read your screenshot clearly... could you type the error?"
A password	It is blurred before saving, and the employee is told to change it.
☒ NEW: a different problem (typed VPN, picture shows a printer jam), in chat	"It shows 'Paper jam...', which looks like a Printer problem. Is it the right screenshot?" Two buttons: wrong screenshot / it's a separate problem.
☒ NEW: a different problem, on the ticket form	A yellow warning on the Review step: "Did you attach the right one? Remove it, or change the category."
During an open ticket, a screenshot of another problem	It asks before acting (this already worked).

The "different problem" check: the AI looks at the typed words and at the screenshot separately. If they clearly point to two different categories (60% sure or more), the bot asks. Setting: `SCREENSHOT_MISMATCH_CONFIDENCE` in `config.py`. Code: `orchestrator.py _screenshot_vs_typed()` and `screenshot_mismatch()`.

2.5 I raised a ticket and logged out. Later the engineer joined. Where do I see it? ☒ NEW

Before: when you logged in again, the chat opened clean, and nothing told you an engineer had joined. Now the Chat page shows a **notice at the top** for every ticket with news, with an **Open that conversation** button:

- 🕒 "TKT-3001 is waiting for the Network Remote Access team."
- 👤 "Ananya from IT is working on TKT-3001."
- ☒ "TKT-3001 has been fixed. Is it working for you?"

Press the button and the full conversation opens, including “ Ananya from Network Remote Access joined the chat.” You simply type there to continue. **My tickets** also always shows every ticket with its whole thread and a reply box. File: [views/employee_chat.py](#) (the notice), [views/my_tickets.py](#).

2.6 How do the engineer's messages reach me?

Like passing notes through one shared notebook. The engineer writes a comment on the Ticket page; it goes into the diary's **messages** table, marked “customer” (employees see it) or “internal” (a work note, only IT). Your chat page re-reads the notebook **every 8 seconds**, so the reply just appears.

Step	File
Engineer writes a comment	views/ticket.py → api.py POST /tickets/{id}/comment
Saved in the diary	servicedesk/services/tickets.py comment() → messages table
Your chat reads it (every 8 s)	views/employee_chat.py → api.py GET /chat/sessions/{id}/messages
Work notes never reach you	the employee calls only ever ask for “customer” messages
You also get a notification (bell; email if switched on)	servicedesk/services/notify.py

2.7 Ratings

Rating	Where	Saved in	Shown on
 /  under each bot answer (and why, after )	Chat	answer_feedback (api.py POST /me/feedback/answer)	Knowledge (Helpful %), Charts tile
★ 1–5 stars when a ticket is fixed	My tickets	ticket_ratings (api.py POST /me/tickets/{id}/rating)	Charts “Satisfaction” tile

3 Fixed? The employee closes the ticket ☒ NEW

The review said: closing should be on the employee's side. Now it is.

Think of a plumber. When he says "done", the job isn't finished until **you** turn on the tap and say "yes, it works". That's how tickets work now.

Who fixed it	What the employee sees	Then
The bot (the employee pressed "Yes, it's fixed")	"Great news! I've closed TKT-... as resolved."	Resolved straight away. Before, it still said "please confirm" and needed a second click in My tickets: fixed.
An engineer pressed Resolve	In the chat: " ☒ Ananya from IT has fixed TKT-.... Is it working for you now? " with two buttons	The ticket waits for the employee (IT sees "Fixed, waiting for the employee").
... employee taps Yes, it's fixed (or types "thanks, works")	"Great, glad it's working! I've closed TKT-... as resolved."	Resolved. Asked to rate under My tickets.
... employee taps No, still not working	"Sorry... I've reopened TKT-... and sent it back to the team."	Reopened, back in the team's queue with an alert.
... employee does nothing	—	Auto-confirmed after 3 days; closed for good after 7.

The same two buttons are on **My tickets**. Files: `servicedesk/services/tickets.py resolve()` (posts the question), `orchestrator.py _on_confirm_fix()` (reads the answer), `views/my_tickets.py`. Timers: `sweep()` in `tickets.py` (3 days / 7 days, `REOPEN_WINDOW_DAYS` in `config.py`).

4 Agent side: queue, alerts, ticket page

4.1 My queue / All queues

A to-do list sorted by danger: P1 first, then whichever promise (SLA) runs out soonest. An agent sees their own team's tickets **plus every P1** (all hands for emergencies). The supervisor sees everything. The list refreshes every 15 seconds.

Filter button	Shows
All	Everything you're allowed to see
Mine	Tickets you took over
Unowned	Nobody has taken them yet
Waiting on a human	The bot handed over and nobody has started
Include resolved tickets	Also show finished ones

Files: `views/queue.py` → `api.py` GET `/queue`. Who-can-see-what: `tickets.py` `can_view()`.

4.2 The alerts (“Unacknowledged alerts”): where are they set?

Like a doorbell that rings louder for emergencies.

Priority	How it rings	Rings again	Supervisor told
P1	Pop-up window + red banner at the top	Every 2 minutes until someone acknowledges	After 10 minutes
P2	Small pop-up (toast) in the corner	—	—
P3 P4	A number badge in the sidebar	—	—

“Unacknowledged” simply means nobody has pressed **Acknowledge** (or **Take over**) yet. Rules: `servicedesk/services/alerts.py` (`LEVEL` = which priority gets which bell). Timings: `P1_REALERT_MINUTES = 2`, `P1_SUPERVISOR_MINUTES = 10` in `config.py`. The screen checks every 10 seconds: `ui/common.py` `alert_center()` and `sidebar_badges()`.

4.3 The Ticket page

Open a ticket by clicking it in **My queue**, or type the number (TKT-3001) in the box at the top. Both work. File: `views/ticket.py`.

- **Take over** = “I’m on it.” The employee sees that you joined.
- **Activity** shows the conversation and work notes by default; bot steps and system changes are one click away.
- **Work notes** tab = secret notes for IT, listed above the box.
- **Checklist** = the must-do steps; **Resolve** stays locked until they’re ticked, a resolution code is chosen and notes are written. Built in `tickets.py` `build_checklist()` from `datasets/agent_playbooks.json`.
- **Handoff package** = everything the bot collected before handing over, shown as readable lines.

5 Bot answers log and success rate ☒ NEW filters

A **report card for the help articles**. Every time the bot sends fix steps, it writes one line: which article, Fix 1 or Fix 2, and later what happened.

5.1 How the success rate is counted

Moment	What is written	Code
The bot sends Fix 1 or Fix 2	A new line in bot_answers, outcome empty (shown as "Waiting for the employee")	<code>orchestrator.py _record_answer()</code>
The employee says "Yes, it's fixed"	outcome = fixed	<code>orchestrator.py _answer_outcome()</code>
The employee says "No"	outcome = not fixed	same
The bot hands over	outcome = escalated	same

Success % = fixed ÷ sent, per article. Only fix steps sent inside a ticket count (Fix 1 and Fix 2). "How do I...?" answers have no ticket, so they are listed but not counted. Low % = rewrite that article first.

5.2 Who sees what ☒ NEW

Before, every engineer saw **every** team's answers (printer, network, password...) and clicking another team's ticket said "Ticket not found". Now:

Who	Sees	Filters
Agent	Only their own team's answers (a printer engineer sees only printer answers)	Search · Team (if they have more than one) · Category · Article · Outcome · Dates
Supervisor	Every team	Search · Team · Agent · Category · Article · Outcome · Dates

The success table at the top is counted from **the same rows** as the list, so picking "VPN & Remote Access" shows only VPN numbers. A line belongs to its ticket's team, or for a "how do I" answer, to the team that owns the category (`datasets/routing_matrix.json`). Files: `views/bot_answers.py`, `api.py` GET `/analytics/bot-answers`, `servicedesk/services/analytics.py` `bot_answers()`.

6 Knowledge and knowledge gaps

6.1 Articles

The list of help articles, with the original (for IT) next to the friendly version (for employees). An agent can write a draft; a meaning check makes sure no step was lost; the **supervisor approves** it, and only then employees see it. ☒ NEW: search, team and category filters, and an agent sees their own team's articles. Files: [views/knowledge.py](#), [servicedesk/services/kb.py](#).

6.2 How is a knowledge gap found?

A knowledge gap is **a real question that no book on the shelf answered well**. When the bot looks for an article, the AI gives every article a score from 0 to 1. If even the best one scores **below 0.50**, the question is written in the gaps list.

Step	File
The AI scores each article (0 to 1)	servicedesk/brain.py <code>ground()</code>
Best score below 0.50 → write it down (passwords removed)	servicedesk/orchestrator.py <code>_gap()</code> → <code>knowledge_gaps</code> table
The 0.50 line	<code>KB_MIN_RELEVANCE</code> in config.py
Grouped by category, most asked first, with the closest existing articles	servicedesk/services/kb.py <code>gaps()</code>
The Knowledge gaps tab (7, 30 or 90 days; ☒ NEW team filter)	views/knowledge.py

So the scoring is done in `brain.py`, and saving the gap is done in `orchestrator.py`. **How to use it:** the top group is the next article to write.

7 Charts C1 to C12

Each chart counts pages in the diary. Nothing is typed in by hand.

All charts are counted in `servicedesk/services/analytics.py` (`kpis()` and `charts()`), sent by `api.py` GET `/analytics/dashboard` and drawn by `views/charts.py`. The filters at the top (dates, team, category, channel, priority) choose which tickets are counted. An agent's charts only count their own teams.

Chart	What it counts, and on what basis	What it's for
Tiles	Open · P1 open · Waiting for a person · Bot fix rate (fixed by bot ÷ all fixed) · SLA met · Reopen rate · Helpful % (👍 ÷ all) · Satisfaction (average stars)	The health of the desk at a glance
C1	Tickets per day, split by channel (Chat / Form / Phone)	Which door people use; busy days
C2	Tickets per category	What breaks most
C3	Per week: fixed by the bot / by a person / handed over / cancelled. Decided by the resolution code on the ticket	Is the bot taking load off the team?
C4	Why tickets went to a person, grouped by keywords in the bot's reason (<code>REASON_GROUPS</code>)	What to teach the bot next
C5	SLA met %, per priority (business hours, paused while waiting on the employee)	Are we keeping our promises?
C6	Hours from opened to fixed, bot vs person	How much faster the bot is
C7	P1–P4 per day	Spot a storm of urgent tickets
C8	How often the bot was unsure of the category (below 50%)	When to improve the categories
C9	Category accuracy = $1 - \text{corrections} \div \text{tickets}$, using the bot's first guess	How often the bot sorts correctly
C10	Success % per help article (same as the Bot answers log)	Which articles to rewrite
C11	Open tickets by age: 0–4 h, 4–24 h, 1–3 days, over 3 days	Find tickets that are going stale
C12	Safety: did dangerous problems reach a person? any restricted action without an identity check? any password left in a chat?	Proof the bot is safe

Target lines on the charts come from `datasets/metrics_acceptance.json`. Dates on Charts and Shift are UTC.

8 Agent performance

A stopwatch that starts when an agent presses **Take over**. Agents see their own row; the supervisor sees everyone. File: `servicedesk/services/analytics.py` `performance()`; screen `views/performance.py`.

Column	How it's counted
Handled	How many times they pressed Take over in the period
Resolved	How many times they pressed Resolve
First response (median hours)	From Take over to their first comment the employee can see (work notes don't count)
Resolution (median hours)	From Take over to Resolve
SLA met %	Of their resolved tickets, how many were within the fix promise (same rule as the timers and C5)
Reopen rate %	How often a ticket they resolved came back
Checklist completion %	Required checklist ticked before Resolve; always 100% because Resolve is locked otherwise
Corrections made	How often they fixed the bot's category, impact, urgency or priority (a good thing)

"Median" = the middle value, so one very slow ticket doesn't spoil the number. The criteria live in the code above; the SLA promises live in `servicedesk/content/service_targets.json`.

9 Shift

The end-of-shift handover page: like a nurse telling the next nurse what happened today.

Part	What it shows	Where it's made
Summary numbers	New today, fixed by bot, fixed by agents, handed over, SLA breaches, AI outages	<code>analytics.py</code> <code>shift_summary()</code>
Summary text	A fixed sentence to copy into the handover channel (no AI writing)	same
Shift checklist (4 items)	Open P1s reviewed · Unacknowledged alerts cleared · SLA-at-risk tickets owned · Handover notes written	<code>analytics.py</code> <code>SHIFT_ITEMS</code>
Live hints next to each item	e.g. "2 open P1: TKT-3004..."	<code>analytics.py</code> <code>shift_status()</code>
Ticks and notes	Saved per person, per day	<code>shift_checklists</code> table, <code>api.py</code> <code>/analytics/shift</code>
The screen		<code>views/shift.py</code>

To change the checklist items, edit `SHIFT_ITEMS` in `servicedesk/services/analytics.py`. It is basic today (one day, UTC); real shift times (e.g. 6–2, 2–10) are a next step.

10 Corrections log and user accounts

10.1 Corrections log (supervisor)

When an agent changes something the bot decided (category, impact, urgency or priority), they must give a reason. Each change is written as "bot said X → corrected to Y, by whom, why". It shows how often the bot is wrong, feeds chart C9, and can be replayed as tests (`python evaluate.py --mode corrections`).

Files: saved by `tickets.py` (`update()` → `store.add_correction()`), listed by `api.py` GET `/corrections`, shown in `views/corrections.py`.

10.2 User accounts (supervisor)

Find a person (name, ID or email), filter by role (employee, agent, supervisor), reset a password, unlock, disable a leaver, change role or team, add accounts one by one or from an HR file. Nothing is deleted.

Files: `views/users.py`, `servicedesk/services/auth.py`. The Bot answers log now uses the same style of filter row.

11 What was fixed after the review

Review point	Before	Now
Bot answers log	Every engineer saw every team's answers; other teams' tickets said "not found"	Own team only; the supervisor filters by team, agent, category, article, outcome, dates
Success rate	Counted over everything, whatever the filter	Counted from the same rows as the list
"Resolved, waiting for confirmation"	Even after the employee said "yes it's fixed", they had to click again in My tickets	The employee's yes closes it; an engineer's fix asks "Is it working?" in the chat with Yes / No
Logged out, engineer joined later	Chat opened clean; no sign of the engineer	A notice at the top with Open that conversation
Wrong screenshot	Only checked during an open ticket	Also on the first message and on the ticket form
Knowledge	Everyone saw everything; no filters	Search, team and category filters; agents see their teams
Ticket page	Work notes looked lost; very long activity list; handoff looked empty	Notes listed in their tab; conversation first; handoff in readable lines
Agent performance SLA	Clock hours (could disagree with Charts)	Same rule as the timers and Charts

All of these are covered by automated tests (119 in total, all passing).

12 Cheat sheet: I want to change...

I want to change...	Open this file
The bot's question wording and buttons	servicedesk/content/field_questions.json
Fix 1 / Fix 2 text, or the "what happens next" text	Knowledge page (approved), or servicedesk/content/kb_employee_drafts.json
Which impact × urgency gives which priority	datasets/priority_matrix.json
Reply and fix promises (SLA hours)	servicedesk/content/service_targets.json
Office hours, holidays, time zone	<code>.env</code> : BUSINESS_HOURS, BUSINESS_DAYS, HOLIDAYS_FILE, TIMEZONE
Which team gets which category	datasets/routing_matrix.json
Form categories, issue types, extra fields	datasets/categories.json , subcategories.json , dynamic_form_fields.json
How many questions / fixes before handing over	<code>config.py</code> : MAX_CLARIFYING_QUESTIONS, MAX_ATTEMPTS
How often P1 rings, when the supervisor is told	<code>config.py</code> : P1_REALERT_MINUTES, P1_SUPERVISOR_MINUTES
When a question counts as a knowledge gap	<code>config.py</code> : KB_MIN_RELEVANCE (0.50)
When a screenshot counts as "a different problem"	<code>config.py</code> : SCREENSHOT_MISMATCH_CONFIDENCE (0.6)
Ticket checklist items	servicedesk/services/tickets.py <code>build_checklist()</code> + datasets/agent_playbooks.json
Shift checklist items	servicedesk/services/analytics.py <code>SHIFT_ITEMS</code>
Escalation reason groups (chart C4)	servicedesk/services/analytics.py <code>REASON_GROUPS</code>
Approved / licensed software list	servicedesk/content/catalog.json

After changing a file, restart the API: these files are read once when it starts.